

Tutorial for the `labmc` software

Authors: Chhavi Jain and Jun Korenaga

June 22, 2026

1 Introduction

The `labmc` software uses a Markov chain Monte Carlo inversion scheme to fit a given rheological model to experimental rock deformation data. The algorithm used here was first described in Korenaga and Karato [2008]. It was later corrected by Mullet et al. [2015] and further optimized by Jain et al. [2019]. Refer to the aforementioned papers for a detailed explanation of the methodology. The steps to install and use the software are described below. The use of the MCMC inversion scheme has been demonstrated by Jain and Korenaga (under review) using synthetic data sets. The synthetic data files and the input scripts used to conduct the inversions described therein and the MATLAB scripts used for processing and visualizing the inversion results are provided with this package.

`labmc` - main executable file

2 Installation

Make sure that a GNU C++ compiler with support for the Standard Template Library (STL) is installed on your system. The MCMC inversion code can be run with or without parallelization. To use the parallel implementation, an MPI library and the corresponding `mpic++` compiler wrapper must also be installed. The *Makefile* provided in the `src` folder is designed to compile the code on Unix-like systems (Linux, macOS, etc.) as well as on Windows systems running WSL, MSYS2/MinGW, or a similar Unix-like build environment.

After downloading the source code from GitHub, open a terminal and navigate to the directory:

```
% cd path_to_src_folder/src
```

To compile the code, type

```
% make
```

By default, this command compiles the parallel version of the code (MPI=1) and requires

the mpic++ compiler wrapper to be available on the system.

To compile the serial version, disable MPI support by setting MPI=0:

```
% make MPI=0
```

To compile the serial version using a specific compiler, such as GNU C++ (g++), type:

```
% make CXX=g++ MPI=0
```

Similarly, to compile the serial version using Clang:

```
% make CXX=clang++ MPI=0
```

Successful compilation should create an executable file named `labmc` (or `labm.exe` on some Windows systems) in the `src` folder.

To remove all object files and executables created during compilation, type

```
% make clean
```

This command restores the source directory to its pre-compilation state.

3 Input and output file formats

3.1 Input data file

The input data file contains the experimental data set to be analyzed. Each line in the input data file must contain the temperature, T in K, pressure, P in GPa, strain rate, $\dot{\epsilon}$ in s^{-1} , and stress, σ in MPa, measured at steady state during an experimental run along with the uncertainties in these measurements, denoted by δT in K, δP in GPa, $\delta \dot{\epsilon}$ in s^{-1} , and $\delta \sigma$ in MPa, respectively, and in the following format:

T	δT	P	δP	$\dot{\epsilon}$	$\delta \dot{\epsilon}$	σ	$\delta \sigma$	run#
-----	------------	-----	------------	------------------	-------------------------	----------	-----------------	------

The ‘run #’ in the format is a numerical identifier assigned to each experimental run and must be specified in the last column. Each experimental run must contain at least two data points.

Depending on the flow law under consideration, information on additional quantities, such as the grain size, d in microns (μm), the water content, C_w in ppm H/Si, the oxygen fugacity, f_{O_2} in MPa, and the melt fraction, ϕ , may also be provided along with the uncertainties in these data. These data must be provided starting from the ninth column and always before the column containing the run number. For example, the grain size d in microns associated with an uncertainty of δd also in microns, is provided as follow,

T	δT	P	δP	$\dot{\epsilon}$	$\delta \dot{\epsilon}$	σ	$\delta \sigma$	d	δd	run#
-----	------------	-----	------------	------------------	-------------------------	----------	-----------------	-----	------------	------

More than one of these additional quantities may be provided simultaneously. For example, in case of a wet sample, the water content C_w with uncertainty, δC_w , both measured in ppm H/Si, can be included in the input data file along with the grain size information as follows,

T	δT	P	δP	$\dot{\epsilon}$	$\delta \dot{\epsilon}$	σ	$\delta \sigma$	d	δd	C_w	δC_w	run#
-----	------------	-----	------------	------------------	-------------------------	----------	-----------------	-----	------------	-------	--------------	------

The order in which the information on the grain sizes and the water content is provided is interchangeable as long as the uncertainty associated each quantity is reported in the column adjacent to the respective quantity.

3.2 Output data file

The code outputs a data file that contains the outcome of each MCMC iteration, which consists of the iteration number, the values of the flow-law parameters accepted by the rejection method in that iteration, and the values of the misfit functions, i.e., the simple χ^2/N [Korenaga and Karato, 2008, Mullet et al., 2015], denoted by χ_s^2/N , and the new χ_J^2/N function [Jain et al., 2019], corresponding to those flow-law parameters. Depending on the flow law considered, the columns of the output data file may vary.

1. For the flow law for diffusion creep:

- under dry conditions,

$$\dot{\epsilon} = A_1 d^{-p_1} \sigma \exp \left(-\frac{E_1 + PV_1}{RT} \right), \quad (1)$$

the flow-law parameters to be estimated are the grain-size exponent, p_1 , the activation energy, E_1 , the activation volume, V_1 , and the scaling coefficient, A_1 . The output file has the following format.

iter#	χ_s^2/N	χ_J^2/N	p_1	E_1	V_1	A_1
-------	--------------	--------------	-------	-------	-------	-------

- under wet conditions,

$$\dot{\epsilon} = A_2 d^{-p_2} C_w^{r_2} \sigma \exp \left(-\frac{E_2 + PV_2}{RT} \right), \quad (2)$$

the water-content exponent, r_2 , is also output. The format of the output file is thus,

iter#	χ_s^2/N	χ_J^2/N	p_2	r_2	E_2	V_2	A_2
-------	--------------	--------------	-------	-------	-------	-------	-------

2. For the flow law for dislocation creep:

- under dry conditions,

$$\dot{\epsilon} = A_3 \sigma^{n_3} \exp \left(-\frac{E_3 + PV_3}{RT} \right), \quad (3)$$

the flow-law parameters to be estimated are the stress exponent n_3 , the activation energy E_3 , the activation volume V_3 , and the scaling coefficient A_3 . The output file has the following format.

iter#	χ_s^2/N	χ_J^2/N	n_3	E_3	V_3	A_3
-------	--------------	--------------	-------	-------	-------	-------

- under wet conditions,

$$\dot{\epsilon} = A_4 C_w^{r_4} \sigma^{n_4} \exp \left(-\frac{E_4 + PV_4}{RT} \right), \quad (4)$$

the water-content exponent, r_4 , is also also output. The format of the output file is thus,

iter#	χ_s^2/N	χ_J^2/N	r_4	n_4	E_4	V_4	A_4
-------	--------------	--------------	-------	-------	-------	-------	-------

3. For a generalized flow law (often used in the rheology of olivine single crystals):

$$\dot{\epsilon} = A_5 f_{O_2}^{m_5} \sigma^{n_5} \exp \left(-\frac{Q_5}{RT} \right), \quad (5)$$

the flow-law parameters are the exponent for oxygen fugacity, m_5 , the activation enthalpy, Q_5 , and the scaling coefficient, A_5 . The output file has the following format.

iter#	χ_s^2/N	χ_J^2/N	n_5	m_5	Q_5	A_5
-------	--------------	--------------	-------	-------	-------	-------

4. For the flow law for dislocation-accommodated grain-boundary sliding (Dis-GBS):

- under dry conditions,

$$\dot{\epsilon} = A_6 d^{-p_6} \sigma^{n_6} \exp \left(-\frac{E_6 + PV_6}{RT} \right), \quad (6)$$

the flow-law parameters to be estimated are the grain-size exponent p_6 , stress exponent n_6 , the activation energy E_6 , the activation volume V_6 , and the scaling coefficient A_6 . The output file has the following format.

iter#	χ_s^2/N	χ_J^2/N	p_6	n_6	E_6	V_6	A_6
-------	--------------	--------------	-------	-------	-------	-------	-------

- under wet conditions,

$$\dot{\epsilon} = A_7 d^{-p_7} \sigma^{n_7} C_w^{r_7} \sigma^{n_7} \exp\left(-\frac{E_7 + PV_7}{RT}\right), \quad (7)$$

the water-content exponent, r_7 , is also also output. The format of the output file is thus,

iter#	χ_s^2/N	χ_J^2/N	p_7	n_7	r_7	E_7	V_7	A_7
-------	--------------	--------------	-------	-------	-------	-------	-------	-------

5. For the flow law for low-temperature plasticity:

- under dry conditions [Kawazoe et al., 2009]

$$\dot{\epsilon} = A_8 \sigma^2 \exp\left(-\frac{E_8 + PV_8}{RT} \left[1 - \left\{\frac{\sigma}{\sigma_P^0 (1 + G'P/G_0)}\right\}^q\right]^s\right), \quad (8)$$

the flow-law parameters to be estimated are the activation energy E_8 , the activation volume V_8 , the Peierls stress, σ_P^0 , and the scaling coefficient A_8 . At this time, the code cannot invert for the exponents q and s . The values of these exponents should therefore be provided in the input parameter (submission) file. The parameter G_0 is the bulk modulus at ambient conditions and G' is its pressure derivative. These parameters are considered pre-defined constants, and their values (appropriate for the mineral olivine) are pre-assigned in the *constants.h* file.

- under dry conditions (Evans and Goetze, 1979)

$$\dot{\epsilon} = A_9 \exp\left(-\frac{E_9 + PV_9}{RT} \left[1 - \left\{\frac{\sigma}{\sigma_P^0 (1 + G'P/G_0)}\right\}^q\right]^s\right). \quad (9)$$

This equation is similar to Eq. (8) except for the pre-exponential σ^2 term, which is ignored in Eq. (9).

- under dry conditions [Mei et al., 2010]

$$\dot{\epsilon} = A_{10} \sigma^2 \exp\left(-\frac{E_{10}}{RT} \left[1 - \left\{\frac{\sigma}{\sigma_P^0}\right\}^q\right]^s\right). \quad (10)$$

This equation is similar to Eq. (8) except that the effect of pressure on σ_P^0 and in the activation enthalpy are not considered here.

- under dry conditions (simplified form)

$$\dot{\epsilon} = A_{11} \exp \left(-\frac{E_{11}}{RT} \left[1 - \left\{ \frac{\sigma}{\sigma_P^0} \right\}^q \right]^s \right), \quad (11)$$

This equation is similar to Eq. (10) except that the σ^2 is not considered here.

The output file for the flow laws (8)–(9) will have the following format,

iter#	χ_s^2/N	χ_J^2/N	σ_P^0	E_i	V_i	A_i
-------	--------------	--------------	--------------	-------	-------	-------

where $i = 8$ or 9 , and for Eqs. (10)–(11), the output format will be,

iter#	χ_s^2/N	χ_J^2/N	σ_P^0	E_i	A_i
-------	--------------	--------------	--------------	-------	-------

where $i = 10$ or 11 .

Note: The output data file reports the values of E_i and Q_i in J mol^{-1} , V_i in $\text{m}^3 \text{mol}^{-1}$, and σ_P^0 in Pa for $i = 1 - 11$.

6. For a composite flow law:

For example, if the parallel operation of diffusion and dislocation creep is considered under dry conditions, expressed mathematically as,

$$\dot{\epsilon} = \dot{\epsilon}_{\text{diff,dry}} + \dot{\epsilon}_{\text{disl,dry}}, \quad (12)$$

then the code will first output the flow-law parameters for diffusion creep followed by those of dislocation creep. Note that the scaling coefficients for both mechanism will be output in the last columns. For this example, the format of the output is,

iter#	χ_s^2/N	χ_J^2/N	p_1	E_1	V_1	n_3	E_3	V_3	A_1	A_3
-------	--------------	--------------	-------	-------	-------	-------	-------	-------	-------	-------

If the input dataset includes more than one experimental run, then the user also has the option to estimate the inter-run bias between different experimental runs. If a dataset is divided into M number of distinct data groups, differentiated from each other by their experimental run number, ‘run#’, then each m^{th} data group is associated with a unique inter-run bias factor, X_m , where $m = 1, \dots, M$. MCMC inversion estimates all M inter-run bias factors denoted by X_1 to X_M . For example, if the input dataset has three data groups, i.e., m ranges from 1 – 3, and is fitted by the composite flow law of equation (12), then the output file will have the following column headings,

iter#	χ_s^2/N	χ_J^2/N	p_1	E_1	V_1	n_3	E_3	V_3	X_1	X_2	X_3	A_1	A_3
-------	--------------	--------------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------	-------

4 Commands

Name **D** - set input data files.

Synopsis

-D[option]*file* [**-d**[option]]

Description

The **-Di** command sets the name of the input data file containing the experimental data set. By default, the code reads the first eight columns of the input data file as T , δT , P , δP , $\dot{\epsilon}$, $\delta \dot{\epsilon}$, σ , and $\delta \sigma$, respectively, and the last column as the run number. If the data file contains additional columns for quantities such as the grain size, the water content, etc., then specifying the appropriate **-d** option allows the corresponding data columns to be read. These additional columns must occur before the column containing the run numbers (Section 3.1). Multiple **-d** options can be specified if more than one additional quantity is included in the data table. The sequence in which each **-d** option is specified must follow the sequence in which the corresponding quantity is reported in the data file. Optionally, the randomized experimental data generated by the code (during data randomization step) can be printed using the **-Do** option.

Options

-Di*input_file*

- Specifies the name (and path) of the input data file that contains the experimental data in the format specified in section 3.1. It is mandatory to set this option.

-Do*ran_data_file*

- Specifies the name (and path) of an output file in which the randomized data is printed. The format of the randomized data is same as that of the input data file. The randomized data generated in each data randomization step is appended to *ran_data_file*.

-d1

- Specifies that the data file contains two additional columns with data on grain sizes and their uncertainties.

-d2

- Specifies that the data file contains two additional columns with data on the water content and the associated uncertainty.

-d3

- Specifies that the data file contains two additional columns with the measurements of the oxygen fugacity and the associated uncertainty.

-d4

- Specifies that the data file contains two additional columns with the measurements of the melt fraction and the associated uncertainty.

Example

Let the input data file '*synthdata.dat*' include observations of the water content as well as the grain sizes, in that order. The input data options in this case would be specified as: **-Disynthdata.dat -d2 -d1**. Additionally, to print the randomized data to '*Rsynthdata.dat*', specify the data options: **-Disynthdata.dat -DoRsynthdata.dat -d2 -d1**.

Name **M** - set the configuration for MCMC sampling.

Synopsis

-Mniter/dn/m/dr

Description

This command sets the following parameters required to configure MCMC sampling:
niter : the total number of iterations to be performed.

dn : the interval at which the MCMC output is printed. For example, if $dn = 1$, all *niter* number of iterations will be printed. If $dn = 100$, the output at every 100th iteration will be printed.

m : the number of trial calculations in Gibbs sampling. For example, if $m = 300$, then in each iteration, 300 trial samples of a particular parameter are considered in the random scan Gibbs sampling step to determine the maximum conditional likelihood.

dr : the interval at which the input experimental data are randomized. For example, if $dr = 100$, then data are randomized after every 100 iterations.

Example

For example, the declaration **-M1000000/10/200/100** specifies that 10^6 MCMC iterations are performed in one simulation and the output is printed at intervals of 10. In each iteration, 200 trial calculations are considered for Gibbs sampling. The input data are randomized within the experimental uncertainties after every 100 iterations.

Name **P** - set the flow laws for creep mechanisms that operate in parallel.

Synopsis

-P[option] [-P[option]] ...

Description

This command is used to specify the flow law for a single creep mechanism. If multiple mechanisms operate in parallel, then the **-P** option can be specified successively and with options specific to each flow law. Each flow law is associated with a distinct set of flow-law parameters (Section 3.2). The a priori range, i.e., the minimum and the maximum permissible values, for each of these flow-law parameters must be provided in the flow-law declaration. If the minimum and the maximum values for a flow-law parameter are equal, then that parameter is assumed to be fixed at the given

value, and it is not estimated in the MCMC simulation. This is true for all flow-law parameters except the scaling coefficient.

The scaling coefficient (or the pre-exponential factor) is treated differently from the other flow-law parameters. It is not estimated by MCMC sampling [see Mullet et al., 2015, Jain et al., 2019]. For each trial model considered in Gibb’s sampling, a “best-fit” value of the scaling coefficient(s) is calculated using either the linear regression technique (in case of a single creep mechanism) or the conjugate gradient search (in the case of a composite rheological model). Therefore, even though the submission script requires the user to provide the logarithm (base 10) of the minimum and maximum values for the scaling coefficient(s), denoted by LA_{min} and LA_{max} , the code does not consider these user-provided minimum and maximum bounds to estimate its “best-fitting” value of the scaling coefficients. Consequently, LA_{min} and LA_{max} can be set to any non-identical pair of values ($LA_{min} \neq LA_{max}$) or both can be set to 0. However, if identical, non-zero values are provided, i.e., $LA_{min} = LA_{max} = LA_0 \neq 0$, then the value of the scaling coefficient is assumed to be fixed at 10^{LA_0} , and the code does not invert for the scaling coefficient. This implies that, to set a fixed value for a scaling coefficient equal to A_0 , designate $LA_{min} = LA_{max} = \log_{10}(A_0)$ in the flow-law declaration. Otherwise, assign $LA_{min} = LA_{max} = 0$.

Options

- Pa** $LA_{1,min}/LA_{1,max}/p_{1,min}/p_{1,max}/E_{1,min}/E_{1,max}/V_{1,min}/V_{1,max}$
- Flow law for dry diffusion creep [Eq. (1)].
- Pb** $LA_{3,min}/LA_{3,max}/n_{3,min}/n_{3,max}/E_{3,min}/E_{3,max}/V_{3,min}/V_{3,max}$
- Flow law for dry dislocation creep [Eq. (3)].
- Pc** $LA_{2,min}/LA_{2,max}/p_{2,min}/p_{2,max}/r_{2,min}/r_{2,max}/E_{2,min}/E_{2,max}/V_{2,min}/V_{2,max}$
- Flow law for wet diffusion creep [Eq. (2)].
- Pd** $LA_{4,min}/LA_{4,max}/n_{4,min}/n_{4,max}/r_{4,min}/r_{4,max}/E_{4,min}/E_{4,max}/V_{4,min}/V_{4,max}$
- Flow law for wet dislocation creep [Eq. (4)].
- Pe** $LA_{5,min}/LA_{5,max}/n_{5,min}/n_{5,max}/m_{5,min}/m_{5,max}/Q_{5,min}/Q_{5,max}$
- Generalized creep law [Eq. (5)].
- Pf** $LA_{6,min}/LA_{6,max}/p_{6,min}/p_{6,max}/n_{6,min}/n_{6,max}/E_{6,min}/E_{6,max}/V_{6,min}/V_{6,max}$
- Flow law for dry Dis-GBS creep [Eq. (6)].
- Pg** $LA_{7,min}/LA_{7,max}/p_{7,min}/p_{7,max}/n_{7,min}/n_{7,max}/r_{7,min}/r_{7,max}/E_{7,min}/E_{7,max}/V_{7,min}/V_{7,max}$
- Flow law for wet Dis-GBS creep [Eq. (7)].
- Pp** $LA_{8,min}/LA_{8,max}/\sigma_{P,min}^0/\sigma_{P,max}^0/E_{8,min}/E_{8,max}/V_{8,min}/V_{8,max}$ -**qqss**
- Flow law for low-temperature plasticity [Eq. (8)]. Note that the values of the exponents q and s are not estimated by MCMC inversion but assumed to be fixed. Therefore, provide only a single value for each exponent. Moreover, the values of the bulk elastic modulus and its pressure derivative are assumed fixed (equal to the values for olivine) and declared in the file *constants.h*.

- Pq** $LA_{9,min}/LA_{9,max}/\sigma_{P,min}^0/\sigma_{P,max}^0/E_{9,min}/E_{9,max}/V_{9,min}/V_{9,max}$ -**qqss**
- Flow law for low-temperature plasticity Eq. (9).
- Pr** $LA_{10,min}/LA_{10,max}/\sigma_{P,min}^0/\sigma_{P,max}^0/E_{10,min}/E_{10,max}$ -**qqss**
- Flow law for low-temperature plasticity Eq. (10).
- Ps** $LA_{11,min}/LA_{11,max}/\sigma_{P,min}^0/\sigma_{P,max}^0/E_{11,min}/E_{11,max}$ -**qqss**
- Flow law for low-temperature plasticity Eq. (11).

In the options given above, n_i , p_i , and r_i denote the stress, the grain-size, and the water-content exponents for the i^{th} creep law, E_i is the activation energy in kJ mol^{-1} , V_i the activation volume in $\text{cm}^3 \text{ mol}^{-1}$, and σ_P^0 is the Peierls stress in GPa. The variable LA_i denotes $\log_{10}(A_i)$, where A_i is the scaling coefficient for the i^{th} flow law. The subscripts ‘min’ and ‘max’ denote the minimum and maximum permissible values for the given flow-law parameter. In the flow laws for low-temperature plasticity, q and s are the exponents for Peierls creep. In the generalized flow law, m is the exponent for oxygen fugacity and Q is the activation enthalpy in kJ mol^{-1} .

Examples

To fit experimental data measured at a constant pressure with the flow law for diffusion creep under nominally anhydrous (dry) conditions, assuming the following a priori bounds on the flow-law parameters: $-1.5 \leq p_1 \leq 3$ and $0 \leq E_1 \leq 600 \text{ kJ mol}^{-1}$, set

-Pa0/0/-1.5/3/0/600/0/0

Note that here, V_1 is set to 0 because all the data are at a constant pressure, meaning that the effect of pressure cannot be constrained using the given data set. If the value of V_1 is known, say $V_1 = 6.5 \text{ cm}^3 \text{ mol}^{-1}$, set the flow law as

-Pa0/0/-1.5/3/0/600/6.5/6.5

To fit experimental data with a composite rheological model which assumes the simultaneous and parallel operation of diffusion and dislocation creep under dry conditions, and consider the following a priori bounds on all of the flow-law parameters: $1 \leq p_1 \leq 4$, $2 \leq n_3 \leq 5$, $100 \leq E_1 \leq 1000 \text{ kJ mol}^{-1}$, $0 \leq E_3 \leq 300 \text{ kJ mol}^{-1}$, and $0 \leq V_i \leq 30 \text{ cm}^3 \text{ mol}^{-1}$ for $i = 1$ and 3 , set the following options:

-Pa0/0/1/4/100/1000/0/30 -Pb0/0/2/5/0/300/0/30

To use the rheological model assumed by Hansen et al. [2011], set the flow laws for dry diffusion creep and dry dis-GBS in parallel, and assume the following fixed values for the flow-law parameters for diffusion creep: $A_1 = 10^{7.6} \mu\text{m}^3 \text{ MPa}^{-1} \text{ s}^{-1}$, $p_1 = 3$, $E_1 = 375 \text{ kJ mol}^{-1}$, and $V_1 = 0$, and the following a priori bounds on the flow-law parameters for dis-GBS: $0.4 \leq p_6 \leq 1$, $1 \leq n_6 \leq 4$, and $200 \leq E_6 \leq 700 \text{ kJ mol}^{-1}$, with $V_6 = 0$, set the following options:

-Pa7.6/7.6/3/3/375/375/0/0 -Pf0/0/0.4/1/1/4/200/700/0/0

Here, only the flow law parameters for dis-GBS are estimated by the MCMC inversion of the experimental data; all the flow-law parameters for diffusion creep, including its scaling coefficient, are assumed fixed.

Name **S** - set the flow laws for creep mechanisms that operate in sequence.

Synopsis

-S[option] **-S**[option] [**-S**[option]] ...

Description

This command is used to specify the flow laws for two or more mechanisms that operate sequentially. The **-S** option can be specified successively to define the flow law for each constituent mechanism. Each flow law is associated with a distinct set of flow-law parameters. The a priori range, i.e., the minimum and the maximum permissible values, for each of these flow-law parameters must be provided in the flow-law declaration. If the minimum and the maximum values for a flow-law parameter are equal, then that parameter is not varied in the MCMC inversion scheme; its value is assumed to be fixed as given. This is true for all flow-law parameters except the scaling coefficients. Refer to the description of the **-P** option to understand how the scaling coefficient is set.

Options

- Sd** $LA_{4,min}/LA_{4,max}/n_{4,min}/n_{4,max}/r_{4,min}/r_{4,max}/E_{4,min}/E_{4,max}/V_{4,min}/V_{4,max}$
- Flow law for wet dislocation creep [Eq. (4)].
- Se** $LA_{5,min}/LA_{5,max}/n_{5,min}/n_{5,max}/m_{5,min}/m_{5,max}/Q_{5,min}/Q_{5,max}$
- Generalized creep law [Eq. (5)].

In the options given above, n_i , m_i , and r_i denote the stress, the oxygen-fugacity, and the water-content exponents for the i^{th} creep law, E_i is the activation energy in kJ mol^{-1} , Q_i the activation enthalpy in kJ mol^{-1} , and V_i the activation volume in $\text{cm}^3 \text{ ml}^{-1}$. The variable LA_i denotes $\log_{10}(A_i)$, where A_i is the scaling coefficient for the i^{th} flow law. The subscripts '*min*' and '*max*' denote the minimum and maximum permissible values for the given flow-law parameter.

Example

Plastic deformation of olivine single crystals often occurs by the sequential operation of creep mechanisms [e.g., Mullet et al., 2015]. To fit experimental data with a composite flow law which assumes the sequential operation of two flow laws of the form of Eq. (5), and the a priori bounds on the flow-law parameters are- $1 \leq n_5 \leq 4$, $0 \leq m_5 \leq 0.5$, and $100 \leq Q_5 \leq 1000 \text{ kJ mol}^{-1}$, set

-Se0/0/1/4/0/0.5/100/1000 **-Se**0/0/1/4/0/0.5/100/1000

Name **F** - fix the value of one flow-law parameter to equal that of another (optional).

Synopsis

-F i_1/i_2

Description

This command sets the value of the i_2^{th} flow-law parameter to always equal that of the i_1^{th} flow-law parameter, where i_1 and i_2 are the sequence in which those particular

flow-law parameters are declared in the rheological model defined by the **-P** and/or **-S** options, not counting the scaling coefficients.

Example

In the following declaration of a rheological model with **-F** option,

-Pa0/0/1/4/100/1000/0/30 -Pb0/0/2/5/0/300/0/30 -F2/5

the 5th flow-law parameter are set to assume the same values as the 2nd parameter. In the given example, a composite rheological model is defined which assumes the parallel operation of diffusion and dislocation creep under dry conditions. For the sequence in which the two flow laws are declared and not taking into account the scaling coefficients, the 2nd flow-law parameter is E_1 for diffusion creep, and the 5th flow-law parameter is E_3 for dislocation creep. According to **-F2/5**, the experimental data are fit with the given composite rheological model assuming that $E_3 = E_1$.

Name **G** - group sticky parameters (optional).

Synopsis

-Gi₁/.../i_n

Description

This option is useful when the MCMC inversion result does not converge even after a sufficiently large number of iterations (*niter*). The flow-law parameters for which the autocorrelation function does not approach 0 even at long lags are called “sticky” parameters. It may be possible to achieve convergence if the sticky parameters are randomized more frequently than the other flow-law parameters. However, the MCMC inversion scheme is so designed that, for sufficiently large values of *niter*, each flow-law parameter is sampled with equal probability. To force it to sample the sticky parameters more frequently, the **-Gi₁/i₂** option makes the MCMC inversion scheme to also randomize the i_2^{th} parameter every time the i_1^{th} parameter is sampled during the random scan Gibbs sampling step and vice versa. This way, the i_1^{th} and the i_2^{th} parameters are randomized twice more frequently than before. More than two sticky parameters can also be grouped together. The positions $i_1, i_2, \dots, i_n$ of the n sticky parameters are counted according to the flow-law declaration, as explained previously in the description of the **-F** command. The **-G** option can also be declared multiple times with different combinations of sticky parameters, if needed.

Example

Say, an experimental data set is analyzed for a composite rheological model of the form of Eq. (12), and the autocorrelation functions for p_1 and V_1 for diffusion creep and V_3 for dislocation creep have values higher than 0.5 even at large lags. One way to attain convergence could be to repeat the data inversion using the **-G** option in the following way,

-Pa0/0/1/4/100/1000/0/30 -Pb0/0/2/5/0/300/-10/20 -G1/2 -G3/6

Here, the 1st and 2nd flow-law parameters together make up one group of sticky parameters

and the 3rd and 6th flow-law parameters form a second group of sticky parameter. For the sequence in which the two flow laws are declared, and not taking into account the scaling coefficients, the 1st, 2nd, and 3rd flow-law parameters are p_1 , E_1 , and V_1 for diffusion creep, and the 6th flow-law parameter is V_3 for dislocation creep. According to the assignment **-G1/2 -G3/6**, the parameters p_1 and E_1 are sampled together and parameters V_1 and V_3 are sampled together during the random scan.

Name **B** - reduce data pairs (optional).

Synopsis

-B*new_runs_filename*

Description

The **labmc** code calculates the misfit function based on the difference between the gradients in the strain rate between different data pairs within an experimental run or data group. If the m^{th} experimental run contains N_m data points, this would lead to $N_m(N_m - 1)/2$ gradient evaluations in that group. If $N_m \geq 20$, this results in over 190 pair evaluations in just one data group, which considerably slows down the conjugate gradient search. To improve efficiency, setting the **-B** option allows the user to divide data from large data groups or runs into smaller data subsets. For example, if a run containing 20 data points is divided into two subgroups of 10 points each, the total number of pair evaluations for the run would reduce to 90, which dramatically speeds up the simulation. We note that redefining data pairs using this option does not introduce new inter-run bias factors in the rheological model, it simply reduces the total pair evaluations within a run. Refer to Jain et al. [2019] for more details. The new user-defined data groups for reducing pair evaluation are saved in the file named (along with its path) *new_runs_filename*. The number of entries in *new_runs_filename* must be equal to the length of the input data record.

Example

The original experimental data contains 40 observations, all measured during a single experimental run and saved in the file *inputdata.dat*. The run number for all 40 observations, entered in the last column of *inputdata.dat*, will be identical in this case, and an inter-run bias factor will not be estimated. However, nearly 800 pair evaluations will occur for every instance of conjugate gradient search. To reduce pairing without introducing inter-run bias factors in the assumed rheological model, the **-B** option is set. Say, 5 data subgroups with 8 data points each are defined, reducing the pair evaluations to a little over 140. The new sequence of run numbers is saved in the file *newruns.dat*, appearing as an array of size 40×1 containing the numbers 1 to 5 repeated 8 times each. The **labmc** code is executed with the following options-

-Di*inputdata.dat* **-B***newruns.dat*

Name **R** - set seed for rand().

Synopsis

-R_{seed}

Description

This command assigns a seed value, *seed*, to set the random number generator function, *rand()*, which initializes the MCMC simulation with a set of random guesses for each of the flow-law parameters. The variable *seed* is a whole number greater than or equal to 0. The same value of *seed* will produce the same initial guesses for a given problem. To perform simulations with different initial guesses, use different values of *seed*.

Name **C** - set the conjugate gradient search (optional, recommended).

Synopsis

-C_{ntrials_cg}

Description

This command sets the use of the conjugate gradient (CG) search for estimating the pre-exponential factor (the scaling coefficient) for each creep mechanism assumed. If **-C_{ntrials_cg}** is not set, the conjugate gradient method will not be used. The conjugate gradient search is useful only when a composite rheological model, i.e., a combination of two or more creep mechanisms, is assumed. If the scaling coefficient for only a single creep mechanism is to be estimated, then the conjugate gradient method is not necessary and setting this option will result in an error.

ntrials_cg : the total number of trial models considered for the CG search.

It must be an integer equal to or greater than 1.

Example

The declaration **-C5** sets the use of the CG search to estimate A_i such that the search is repeated 5 times with distinct random initial guesses.

Name **I** - to resume or extend a simulation (optional).

Synopsis

-I_{previous_output_file}

Description

This command allows the user to resume an MCMC simulation which was prematurely terminated, i.e., stopped before it completed the assigned *niter* number of iterations, or extend the simulation beyond the assigned value of *niter*. To restart or extend, provide the full path to and name of the output file for the simulation that needs to be resumed or extended. The code reads only the last line of the output file named *previous_output_file* provided by the user, sets the iteration counter to the next iteration number, and assumes the corresponding values of the flow-law parameters as the last accepted model.

Example

Say, an MCMC simulation is terminated after 50,000 iterations even though the simulation was run with the following option: **-M1000000/1/200/100**, i.e., $niter = 10^6$. The output from the 50,000 iterations is saved in the file *./output.dat*. To restart this simulation from iteration number 50,001 and run till $niter = 10^6$, use the same input options as before with the additional command:

-I“./output.dat”

In the above example, modifying the input to the **-M** option to **-M1500000/1/200/100** will extend the total number of iterations by 500,000.

Name **V** - sets verbose mode (optional).

Synopsis

-V

Description

This option sets the verbose mode.

5 Workflow

In general, the **labmc** code is implemented as follows.

For example, we wish to analyze dataset *input.ex1.dat*, located in a subdirectory named *./input*. The input data contains strain rates measured at variable stresses and grain sizes but at constant values of temperature and pressure, all under dry conditions. All data belong to a single experimental run. We aim to fit these data with the flow law for dis-GBS creep [Eq. (6)]. The path to the executable file for our **labmc** code is: *../src/labmc*, and we wish to save the output file in a subdirectory named *./output* under the filename *output.ex1.dat*. The MCMC simulation should be run for 10^6 steps, data must be randomized after every 100 iterations, and in each iteration, Gibb’s sampling must consider 200 trial calculations.

To analyze the given data using the **labmc** code, we must invoke the **-Pf** option to set our model equation. Because all data are at a constant temperature and pressure, E_6 and V_6 cannot be constrained, therefore we set them to 0 in our assumed flow law. We assume the following wide a priori bounds on the flow-law parameters to be estimated: $-5 \leq p_6 \leq 5$ and $0 \leq n_6 \leq 5$. Moreover, as per the information given, the **-M** option is set with the following parameter values- $niter = 10^6$, $m = 200$, and $dr = 100$. Assuming this is the first MCMC simulation for the given data and model, we assign $seed = 0$ and output the result after every MCMC iteration, i.e., $dn = 1$. The results of all 10^6 MCMC iterations will be needed to calculate the autocorrelation function and test for convergence. The simulation is run by typing the following (in a bash terminal),

```
../src/labmc -Di"./input/input.ex1.dat" -d1 -M1000000/1/200/100 -V -R0 \
-Pf0/0/-5/5/0/5/0/0/0/0 > ./output/output.ex1.dat
```

A second simulation for the same inverse problem can be run using a different value of *seed*, say *seed* = 1, and its output may be written after every 100 iteration, i.e., *dn* = 100. Such a simulation is implemented by the following command:

```
../src/labmc -Di"./input/input.ex1.dat" -d1 -M1000000/100/200/100 -V -R0 \
-Pf0/0/-5/5/0/5/0/0/0/0 > ./output/output.ex1.1.dat
```

Next, suppose we wish to analyze the same dataset for a composite rheological model that includes the parallel operation of diffusion and dislocation creep. In this case, we must set the **-C** option to use the conjugate-gradient search with *ntrials_{cg}* = 3 (say). The MCMC simulation is run using the command,

```
../src/labmc -Di"./input/input.ex1.dat" -d1 -M1000000/100/200/100 -V -R0 -C3 \
-Pa0/0/-5/5/0/0/0/0/0 -Pb0/0/0/5/0/0/0/0 > ./output/out.ex1_composite.0.dat
```

More functionalities of the code are demonstrated in the examples discussed in the next section.

6 Sample scripts

We demonstrate the use of the **labmc** code to analyze experimental data with the help of a few representative examples (Cases S1 – S9) [Jain and Korenaga (under review)]. We provide here the input data files and submission scripts used to run the **labmc** code for each example and the MATLAB scripts we use to analyze the MCMC inversion results.

6.1 Shell scripts

The folders for Cases S1–S9 include the following shell scripts to run the inversion:

- Case S1 includes two submission scripts to analyze data in *dataS1.dat*. The data comprises three experimental runs, all at the same temperature and pressure. The submission script *inv_S1g.sh* is used to run the **labmc** code to fit these data with the flow law for dis-GBS creep and estimate p_5 , n_5 , and A_5 , assuming $E_5 = V_5 = 0$ and without taking inter-run bias into account. Script *inv_S1f.sh* fits data with the flow law for diffusion creep and estimates p_1 and A_1 , assuming $E_1 = V_1 = 0$ and no inter-run bias.

- Case S2 includes one submission script to analyze data in *dataS2.dat*. The dataset contains 8 experimental runs that together encompass a range of temperatures, pressures, stresses, and grain sizes. The submission script *inv_S2f.sh* uses the `labmc` code to fit the dataset with the flow law for diffusion creep and estimate p_1 , E_1 , V_1 , and A_1 . Inter-run bias is not assumed.
- Case S3 includes two submission scripts to analyze data in *dataS3.dat*. The dataset comprises three experimental runs, all at the same temperature and pressure. The submission script *inv_S3f.sh* uses the `labmc` code to fit the data with the flow law for diffusion creep and estimate p_1 and A_1 without considering inter-run bias between the runs and assuming $E_1 = V_1 = 0$. Script *inv_S3fX.sh* also fits the data with the flow law for diffusion creep but takes inter-run bias into account. It estimates p_1 , A_1 , X_1 , X_2 , and X_3 .
- Case S4 includes three submission scripts to analyze data in *dataS4.dat*. The dataset comprises three experimental runs, all at the same pressure. All the data within each run are at the same temperature, but the temperature differs amongst the three runs. The submission script *inv_S4f.sh* fits data with the flow law for diffusion creep without considering inter-run bias. It is used to estimate p_1 , E_1 , and A_1 assuming $V_1 = 0$. Script *inv_S4fX.sh* also fits data with the flow law for diffusion creep but takes inter-run bias into account. It estimates p_1 , E_1 , A_1 , X_1 , X_2 , and X_3 . Script *inv_S4fX-wb.sh* is identical to *inv_S4fX.sh* except that it assumes wider a priori bounds on E_1 .
- Case S5 includes two submission script to analyze data in *dataS5.dat*. The dataset belongs to a single experimental run, and all the data are at the same temperature and pressure. The submission script *inv_S5g.sh* fits data with the flow law for dis-GBS creep. It is used to estimate p_5 , n_5 , and A_5 , assuming $E_5 = V_5 = 0$. Script *inv_S5s.sh* fits the data with the flow law for dislocation creep and estimates n_3 and A_3 , assuming $E_3 = V_3 = 0$.
- Case S6 includes one submission script to analyze data in *dataS6.dat*. The dataset contains 8 experimental runs that together sample a range of temperatures, pressures, stresses, and grain sizes. The submission script *inv_S6s.sh* uses the `labmc` code to fit these data with the flow law for dislocation creep and estimates n_3 , E_3 , V_3 , and A_3 . It does not account for possible inter-run bias between runs.
- Case S7 includes four submission scripts to analyze data in *dataS7.dat*. The dataset belongs to a single experimental run for which temperature and pressure are fixed. The submission script *inv_S7g.sh* uses the `labmc` code to fit these data with the flow law for dis-GBS creep assuming $E_5 = V = 5 = 0$ and estimates p_5 , n_5 , and A_5 . The scripts *inv_S7f.sh* and *inv_S7s.sh* fit the same dataset with the flow law for diffusion creep and dislocation creep, respectively, with the former assuming $E_1 = V_1 = 0$ and the latter $E_3 = V_3 = 0$. The former estimates p_1 and A_1 , and the latter estimates n_3 .

and A_3 . Finally, script *inv_S7fs.sh* is used to fit the data with a composite flow law that considers the parallel operation of diffusion and dislocation creep. The flow-law parameters p_1 , n_3 , A_1 , and A_3 are estimated, assuming $E_1 = V_1 = E_3 = V_3 = 0$. This script uses the conjugate search [option **-C**] to determine the scaling coefficients A_1 and A_3 . Moreover, we also demonstrate the use of the **-B** option to reduce the number of data pairs considered.

- Case S8 includes two submission scripts to analyze data in *dataS8.dat*. The dataset comprises 5 experimental runs and explores a range of temperatures, pressures, stresses, and grain sizes. The submission script *inv_S8fsX.sh* uses the **labmc** code to fit the dataset with the composite flow law 12 that assumes the parallel operation of diffusion and dislocation creep, and it considers the possibility of inter-run biases between all the runs. The flow-law parameters p_1 , E_1 , V_1 , n_3 , E_3 , V_3 , A_1 , and A_3 are estimated along with 8 inter-run bias factors X_1 to X_8 . This script uses the conjugate search option [**-C**]. Script *inv_S8fsX.sticky.sh* performs the same inversion but with setting sticky parameters using the **-G** option. The following parameter pairs are considered sticky: (p_1, V_1) ; (E_1, E_3) , and (V_1, V_3) . We note that the results of the inversion described by the script *inv_S8fsX.sh* do not indicate the presence of sticky parameters. We set sticky parameters only for the purpose of demonstration. The results presented in Jain and Korenaga (in prep.) correspond to the inversions assuming sticky parameters.
- Case S9 includes two submission scripts to analyze data in *dataS9.dat*. The dataset comprises 12 experimental runs under wet conditions and explores a range of temperatures, pressures, stresses, grain sizes, and water contents. The submission script *inv_S9fsX.sh* uses the **labmc** code to fit the dataset with a composite flow law similar to Eq. 12 but for wet rheologies, i.e., it assumes the parallel operation of diffusion and dislocation creep under wet conditions. The possibility of inter-run biases between all the runs is also considered. The flow-law parameters p_2 , r_2 , E_2 , V_2 , n_4 , r_4 , E_4 , V_4 , A_2 , and A_4 are estimated along with 12 inter-run bias factors X_1 to X_{12} . This script uses the conjugate search option [**-C**].

6.2 Matlab scripts

The following post-processing steps are performed in MATLAB:

1. Checking for convergence of an MCMC simulation by calculating the autocorrelation function for each of the estimated flow-law parameters, excluding the scaling coefficients. This task is performed and the results visualized using the MATLAB script *ACF_output.m*.
2. Resampling the MCMC output based on the autocorrelation function to obtain independent models. If more than one MCMC simulation is conducted for the same data

and model (parallel runs), then the resampled output from all the runs are compiled and analyzed together.

3. The MCMC output contains the values of the normalized misfit function χ_J^2/N , which is not straightforward to interpret. The more intuitive misfit function, χ_M^2/N , is calculated from the MCMC output.
4. The a posteriori probability distributions of the χ_M^2/N function and each of the flow-law parameters and inter-run bias factors (if any) estimated by the MCMC inversion of data is visualized.
5. The mean and the standard deviation of each of the flow-law parameters is determined (assuming a normal distribution). The correlation coefficient between each pair of model parameters is also calculated.
6. Finally, the fit between the observed strain rates and those predicted using the mean model parameters is visualized. Optionally, the confidence intervals about the mean predictions can also be plotted.

The steps 2 to 6 are performed by the script *datafit_dry_flowlaw.m*, for inversions assuming dry flow laws, and *datafit_wet_flowlaw.m*, for inversions with wet flow laws.

An additional script *perturb_MCMC_models.m* is also provided that allows users to construct an ensemble of permissible flow laws by perturbing the MCMC-derived mean values of the flow-law parameters for a given case within their respective uncertainty bounds.

Accessory functions needed to implement the above-mentioned MATLAB scripts are also provided in the folder “MATLAB_scripts”.

References

- L. Hansen, M. Zimmerman, and D. L. Kohlstedt. Grain boundary sliding in San Carlos olivine: Flow law parameters and crystallographic-preferred orientation. *Journal of Geophysical Research: Solid Earth*, 116(B8), 2011.
- C. Jain, J. Korenaga, and S.-i. Karato. Global analysis of experimental data on the rheology of olivine aggregates. *Journal of Geophysical Research: Solid Earth*, 124(1):310–334, 2019.
- T. Kawazoe, S.-i. Karato, K. Otsuka, Z. Jing, and M. Mookherjee. Shear deformation of dry polycrystalline olivine under deep upper mantle conditions using a rotational Drickamer apparatus (RDA). *Physics of the Earth and Planetary Interiors*, 174(1-4):128–137, 2009.
- J. Korenaga and S.-i. Karato. A new analysis of experimental data on olivine rheology. *Journal of Geophysical Research: Solid Earth*, 113(B2), 2008.

- S. Mei, A. Suzuki, D. Kohlstedt, N. Dixon, and W. Durham. Experimental constraints on the strength of the lithospheric mantle. *Journal of Geophysical Research: Solid Earth*, 115(B8), 2010.
- B. G. Mullet, J. Korenaga, and S.-i. Karato. Markov chain Monte Carlo inversion for the rheology of olivine single crystals. *Journal of Geophysical Research: Solid Earth*, 120(5): 3142–3172, 2015.